

Unix/Bash

Plan

Unix/Bash

- Introduction

- Shell - Interpréteur de commandes

- Arborescence et système de fichiers

- Substitutions et variables

- Mécanismes d'entrées/sorties

Unix/Bash

Introduction

Un modèle en couche

- une couche matérielle, une couche système d'exploitation et une couche application

Un ordinateur

Machine programmable servant au traitement de l'information (les données).

Est constitué de :

- Mémoire centrale : stocke le programme en exécution dans la mémoire ainsi que les données en cours de traitement
- Processeur(s) : unité de traitement exécutant les instructions du programme et modifiant les données
- périphériques d'entrée/sortie : tout ce qui permet d'échanger avec l'extérieur : souris, clavier, etc.

Système d'exploitation (SE)

Le système d'exploitation est le logiciel de base qui fait le lien entre le matériel, les applications et les utilisateurs.

Exemples : Windows, macOS, Linux, Android, ...

- SE mono-processus, mono-utilisateur : exécute un programme à la fois avec un seul utilisateur (DOS)
- SE multi-processus, mono-utilisateur : plusieurs tâches simultanées correspondant à des programmes différents. Répartition des ressources à tour de rôle entre les applications. (Windows 95, Android 5)
- SE multi-processus, multi-utilisateurs. Plusieurs utilisateurs peuvent lancer en même temps leurs programmes sur la même machine. (exemple : Unix)

Les stations de travail à l'ENSEIRB-MATMECA sont multi-processus, multi-utilisateurs

processus : toute occurrence en mémoire d'un programme en cours d'exécution

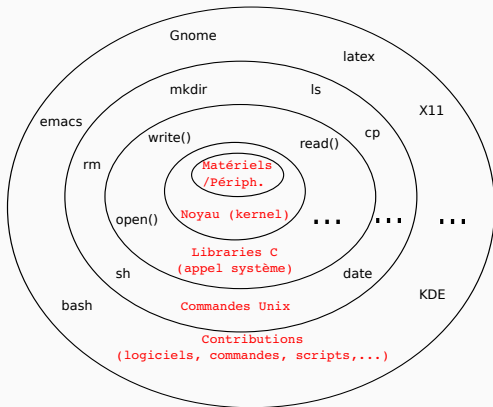
- Une famille de systèmes : Linux, BSD, macOS pour certains aspects, systèmes serveurs, ...
- Plusieurs distributions Linux : Debian, Ubuntu, Fedora, [AlmaLinux](#), ...
- Une culture commune : fichiers, processus, utilisateurs, permissions, outils combinables
- Un même environnement utilisable en local ou à distance

Quelques caractéristiques d'UNIX :

- système à temps partagé : exécution concurrente de processus, ordonnancement
- système multi-utilisateur : compte, connexion (login/password), environnement personnalisé
- partage des ressources : arborescence globale des fichiers, mécanismes de protection
- gestion des communications : entre processus, entre utilisateurs
- simple, portable, extensible

Organisation générale

Un noyau central gère le système de fichiers, les processus, la mémoire, les communications et les périphériques. Autour, on trouve des commandes et des logiciels.



- S5/S6 : les couches externes
- S7/S8 : les couches internes

Sous Linux, l'interface graphique est une couche au-dessus du système d'exploitation. Le terminal reste utilisable même sans interface graphique.

- Plusieurs environnements graphiques existent : [Xfce](#), Gnome, KDE, ...
- Une machine distante peut être utilisée sans écran ni souris.
- Certaines applications graphiques peuvent être lancées à distance avec `ssh -X`, si la configuration le permet.
- Pour beaucoup de tâches de développement, le terminal est l'interface la plus directe.

Compte utilisateur et protection

- Se connecter à un système multi-utilisateur nécessite un compte.
- À chaque compte sont associés un login, un mot de passe et un répertoire personnel (home directory).
- Le répertoire personnel est noté ~ et sert de point de départ.

Rappel sur la protection des comptes/mots de passe

- **Ne prêtez jamais** votre compte ou votre mot de passe.
- **Vous êtes responsable** de ce qui est fait avec votre compte.
- Un échange de fichiers se fait avec les outils prévus : Git, dépôt, archive, partage contrôlé.
- Votre mot de passe doit résister aux essais automatisés : pas de nom, pas de mot usuel, pas de variante évidente.

Première manipulation

Première connexion

- Démarrer la machine et choisir l'item 1 (démarrage d'AlmaLinux), le noyau est chargé en mémoire, c'est la phase de boot
- Un fois terminé la phase de boot : le système d'exploitation démarre et vous obtenez une invite de commande : connectez vous (login/password)
- mot de passe validé via `http://ccc.ipb.fr`
- si ok : le window manager se lance (Xfce), bienvenue sous Linux, vous êtes dans votre *home directory*
- chercher dans *Applications/Accessoires* et lancer l'application *Terminal* (interpréteur de commande/console) (Alt-F2)
- vérifier que vous êtes chez vous en tapant la commande `pwd` (print name of current/working directory) (vous devez voir quelque chose de la forme `/truc/machin/votrelogin`)
- lancer dans la console le navigateur internet `firefox`
- vérifier que votre connexion à internet fonctionne

Le système de fenêtrage

Quelques commandes **clavier** de base pour bien utiliser son environnement (à remplir)

<i>Raccourci</i>	<i>Fonctionnalité</i>
	Lancer une application
	Maximiser la fenêtre active
	Minimiser la fenêtre active
	Changer de fenêtre active
	Fermer la fenêtre active
	Se déplacer sur le bureau en dessous
	Déplacer la fenêtre active sur le bureau en dessous
	Déplacer la fenêtre active sur le bureau

Unix/Bash

Shell - Interpréteur de
commandes

Quelques définitions

- **shell** : c'est un interpréteur de commandes pour interagir avec le noyau (shell : coquille qui entoure le noyau).
- **interpréteur de commande** : programme (processus) qui fournit une interface texte entre l'utilisateur et le noyau
 - **rôle** : lire la ligne de commande et exécuter les actions associées
- **bash** : Bourne-Again shell
C'est le programme shell par défaut dans la plupart des environnements.

Remarque

Dans la suite du cours, nous mélangerons souvent les termes shell et bash pour désigner la même chose. Mais il existe plusieurs implémentations de shell différentes comme : `sh`, `csh`, `zsh`, `ksh`, `cmd.exe` ...

Le shell - Fonctionnement

Fonctionnement général – le shell exécute la boucle ∞ suivante :

POUR toujours FAIRE :

- | Affichage du prompt (prêt à lire) et attente d'une commande
- | Vérification de la syntaxe
- | SI la syntaxe est correcte ALORS :
 - | exécuter la commande
 - | attente de la fin de la commande en cours
- | SINON
 - | afficher un message d'erreur

Votre prompt est la chaîne de caractères, appelée également **invite de commande** qui s'affiche dans votre terminal avant que vous ne tapiez une commande.

Traditionnellement, il se termine par un \$ pour un utilisateur normal, et par un # quand on est connecté en tant que superutilisateur. On verra plus tard comment le modifier.

Qu'est-ce qu'une commande shell ?

- C'est une suite de mots-clés séparés par le caractère de séparation : **l'espace**
Ne pas mettre d'espace dans les noms de fichiers
- Toute commande Unix a la même syntaxe : (séparateur : l'espace)
`cmd options arguments`
`ls -l Documents`
- `enter` lance l'interprétation de la commande
- résultat de la commande dans le terminal
- `options` est une suite de mot clés de la forme :
`-o` ou `-o valeur` pour les options *courtes*
`--option` ou `--option=valeur` pour les options *longues*
- Remarques :
 - Lors de la description d'une commande, toute option ou paramètre optionnel sera affiché entre crochets `[-v]`, et toute option ou paramètre obligatoire est affiché sans crochets.
 - Souvent, seul le nom de commande est obligatoire.

Quelques commandes de base

Entrer dans le sous-dossier divers du sous-dossier autres :

```
$ cd autres/divers
```

Savoir dans quel dossier on se trouve :

```
$ pwd  
/home/login/autres
```

Lister le contenu du dossier courant :

```
$ ls  
dossier1 dossier2 fichier.txt
```

Lister le dossier courant avec les détails :

```
$ ls -la
```

Exercice : premier contact

Mission web associée :

<https://cours-mf.gitlabpages.inria.fr/if104/unix/missions-terminal-fichiers/>

1. Ouvrez un terminal
2. Testez `echo hello world !`
3. Observez votre position avec `pwd`
4. Listez le contenu avec `ls`, puis `ls -l`
5. Déplacez-vous avec `cd Documents`, puis `cd ..`
6. Comparez `ls`, `ls -l` et `ls --reverse`

Questions à se poser

Quelle est la commande ? Quelles sont les options ? Quel est le répertoire courant avant et après la commande ?

La commande `man` :

- `man <cmd>` affiche le manuel d'une commande.
- Les options longues et courtes y sont documentées.
- Certaines pages existent dans plusieurs sections : `man sleep` et `man 3 sleep`.

Dans `man`

`q` quitte, espace avance d'un écran, `/<motif>` cherche, `n` passe au résultat suivant.

Ressources utiles : <http://www.explainshell.com/> et
<http://www.grymoire.com/Unix/index.html>

Exercice : manuel

Essayons de nous familiariser avec `man` :

1. Comment peut-on accéder à la documentation de la commande `man` ?
2. Trouver quelles sont les sections des pages de manuels ?
3. A l'aide des pages de manuels, trouvez :
 - le nom complet de l'option `-a` utilisée précédemment avec `ls`
 - même chose avec l'option `-l`
 - inversement, quel est l'option courte équivalente à `--reverse` ?
 - Peut-on afficher les tailles des fichiers de façon plus lisible avec la meilleure unité possible ? Si oui, comment ?

Pour les curieux, ouvrez le `man` de `gcc`. Avec la touche espace, essayez d'atteindre le bas du texte. Bonne chance !

Historique et raccourcis utiles

Le shell intègre des fonctions de gestion de l'historique des commandes exécutées

- C-r (reverse search) : recherche avec motif
- C-p (previous) : commande précédente (up)
- C-n (next) : commande suivante (down)
- history : affiche l'historique des commandes
- !num : relance la commande de numéro donné dans l'historique

Quelques raccourcis utiles pour le shell

- C-k (kill) : coupe et mémorise le texte depuis la position du curseur jusqu'à la fin de la ligne
- C-y (yank) : insère le texte précédemment supprimé
- C-a : début de ligne
- C-e : fin de ligne
- **Tab : active la complétion (à utiliser sans modération ! !)**

Exercice : quelques commandes en vrac

Objectif : observer, deviner, puis vérifier avec le manuel.

- Essayer les commandes suivantes et interpréter les résultats :

dite	cal 10 2012
date	who
whoami	uname -a
pws	uptime
pwd	top (comment quitter cette commande?)
cal	time sleep 5
cal 2012	history

- Grâce à la complétion rechercher une commande qui nettoie l'écran (c1...) vérifier à l'aide du manuel
- Grâce à la complétion rechercher une commande qui permet de quitter le shell (ex...). Vérifier à l'aide du manuel

Réflexe

Une erreur de commande est une information : le shell explique souvent ce qu'il n'a pas compris ou ce qu'il n'a pas trouvé.

Enchaînement et combinaison de commandes shell

Exécuter plusieurs commandes

L'enchaînement de plusieurs commandes se fait avec ;

```
cmd1; cmd2; cmd3
```

Le groupement de commandes se fait à l'aide des parenthèses

```
(cmd1; cmd2)
```

Utiliser le code de retour

On peut combiner les commandes en testant leur réussite ou leur échec :

```
cmd1 || cmd2
```

```
cmd1 && cmd2
```

À ne pas confondre avec

```
cmd1 & cmd2
```

```
cmd1 | cmd2 (Voir la section redirections)
```


Exercice : tâche de fond

Quelle est la différence avec les commandes suivantes ?

- `ls . & ls ..`
- `ls /toto & ls ..`

On lancera plusieurs fois la commande pour bien observer le comportement.

1. Lancez la commande `emacs`.
2. Est-ce que vous pouvez exécuter une nouvelle commande dans le shell ?
3. Fermez `emacs` (`Alt-F4`).
4. Dédurre des questions précédentes une solution qui permette de lancer `emacs` tout en récupérant le contrôle du terminal.

Gestion des processus

processus : toute occurrence en mémoire d'un programme en cours d'exécution. Il est identifié par un numéro appelé le **PID**

Contrôler le processus courant

- Lancer une commande en **tâche de fond** : `cmd &`
- `C-z` : **endort le processus en cours**
- `C-c` : **termine le processus en cours**
- `C-d` : quitte le shell en cours (`=exit`)

Manipuler les jobs du shell

- `jobs` : liste les jobs connus du shell
- `bg [n]` : réveille un job en tâche de fond (background)
- `fg [n]` : remet un job au premier plan (foreground)

PID et numéro de job

Le numéro utilisé par `fg` et `bg` n'est pas le *PID*. Le *PID* s'obtient avec `ps` ou `echo $!` pour la dernière commande lancée.

- Obtenir la liste des processus : `ps`
- Obtenir le *PID* de la dernière commande lancée en tâche de fond : `echo $!`
- Demander l'arrêt d'un processus spécifique
`kill pid`
- Forcer l'arrêt d'un processus spécifique
`kill -KILL pid`

Réflexe

Pour terminer un programme proprement, on essaie d'abord de le fermer normalement ou avec C-c. `kill -KILL` force l'arrêt.

Exercice : gestion des tâches 1

- Lancer `emacs`
- Terminer le programme par un raccourci clavier et reprendre la main
- relancer la commande ; endormir le processus avec un raccourci clavier pour récupérer la main
- Revenez sur `emacs` et essayer de taper du texte. Est-ce que c'est possible ?
- Revenez sur le terminal, et repassez `emacs` au premier plan. Est-ce que son contenu à changé ?
- Endormez de nouveau `emacs`, puis faites-le passer en tâche de fond
- Essayez à nouveau d'interagir avec `emacs`
- Revenez sur le terminal et essayez de tuer le processus de `emacs` depuis la console.

Exercice : gestion des tâches 2

- Lancez la commande `ls -l&`
- Que se passe-t-il ? Pouvez-vous l'expliquer ?
- Tapez la commande `firefox & oneko &`
- Reprenez la main sur firefox en premier plan
- Terminez firefox avec un raccourci clavier
- Lancez `firefox`, puis endormez le
- Lancez `oneko`, puis endormez le
- Faites exécuter `firefox` en arrière-plan, sans modifier l'état de `oneko`.

Pour aller plus loin, vous pouvez regarder l'exercice sur la généalogie de Gimli :
https://cours-mf.gitlabpages.inria.fr/if104/advanced/02_genealogie/

Unix/Bash

Arborescence et système de
fichiers

Les fichiers sont structurés sous la forme d'un arbre dont la racine est `/`.

Organisation

- les noeuds sont des répertoires ;
- les feuilles sont des fichiers réguliers ;
- chaque fichier est désigné par un chemin dans l'arbre.

Petit lexique

- fichier : file
- répertoire : directory
- chemin : path
- répertoire de connexion : home directory (`~`, *tilde*)
- répertoire courant : working directory (`pwd`)

Sous Unix, tout est fichier

Sous Unix, beaucoup d'objets du système sont manipulés comme des fichiers.

- fichier régulier : texte, code source, commande exécutable, ...
- répertoire : fichier contenant des noms d'autres fichiers
- fichier spécial : périphérique, tube de communication, socket, ...

Vocabulaire

Dans la suite, si on ne précise pas fichier *régulier*, le mot fichier désigne n'importe quel noeud de l'arbre.

Arborescence standard

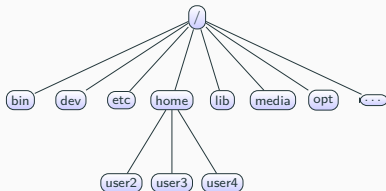
Dans la majorité des cas (variantes possibles selon les versions)

- les répertoires de fichiers exécutables : `/bin`, `/usr/bin`, `/usr/local/bin`
- répertoires de bibliothèques : `/lib`, `/usr/lib`, `/usr/local/lib`
- pages de manuel : `/usr/man`
- répertoire relatif à l'administration `/etc` ex. `/etc/passwd` (contenant des infos sur les utilisateurs)
- divers : `/tmp`, `/dev` (fichiers spéciaux : drivers)

Chaque processus lancé possède un répertoire courant : sa position dans l'arborescence

- Ex. lors de la connexion, le répertoire courant du shell est le répertoire de l'utilisateur (home directory ou `~`)
- Pour modifier cette position courante on utilise la commande : `cd` (change directory)

Arborescence standard



ATTENTION: ici, les utilisateurs sont dans un répertoire `/net/i/` (au lieu de `/home`).

Notion de chemin

Un chemin identifie un fichier dans le système.

Formattage

Il correspond à un déplacement dans l'arborescence et s'écrit comme une suite de noms séparés par / : `/usr/share/fonts`

Deux parties

- le `dirname`, ou préfixe, précède le dernier / ;
- le `basename` suit le dernier / ;
- le préfixe est **toujours** le chemin d'un répertoire.

Exemple : dans `/usr/share/fonts`, le préfixe est `/usr/share`.

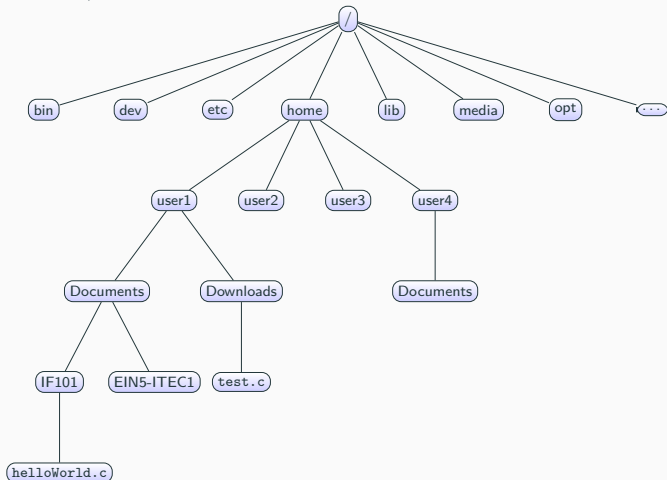
Noms particuliers

`.` désigne le répertoire courant et `..` son parent. D'où le `cd ..` dans l'exercice page 15.

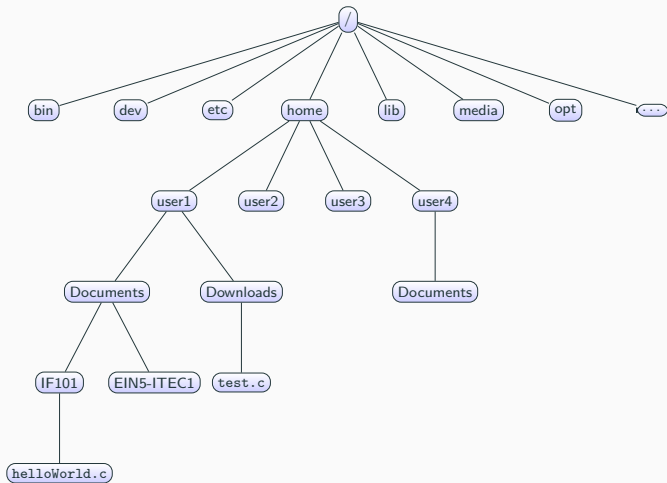
Chemins absolus

Le point de départ **ne dépend pas de la position courante** (working directory) dans le système de fichier. Il est défini quel que soit l'endroit où on se trouve dans l'arborescence. Il commence par :

- / : La racine du système de fichier
- ~ : Alias/Raccourci vers le répertoire home de l'utilisateur courant
- ~toto : Alias/Raccourci vers le répertoire home de l'utilisateur toto



Chemins absolus

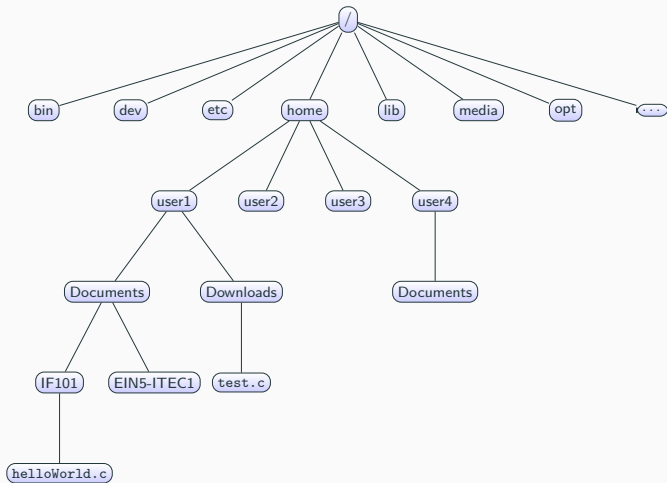


- Je suis l'utilisateur user1 et je suis dans mon répertoire de travail. Pour aller dans le répertoire IF101 je fais :

`cd /home/user1/Documents/IF101`

ou : `cd ~/Documents/IF101`

Chemins absolus

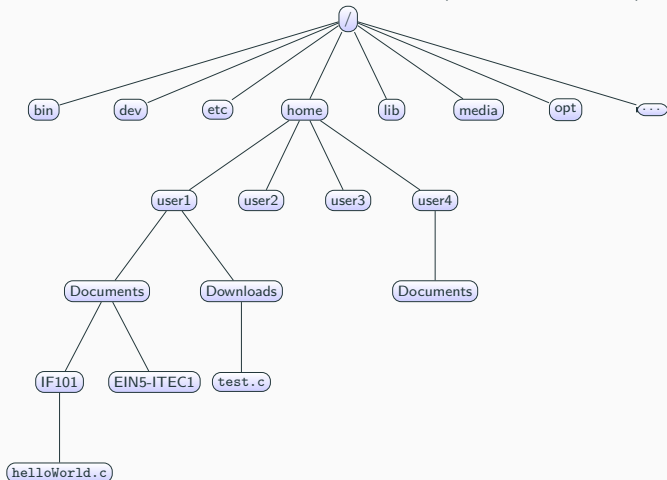


- Je suis l'utilisateur user1 et je suis dans mon répertoire de travail. Pour aller dans le répertoire Documents de user4 je fais :
 `cd /home/user4/Documents/`
ou : `cd ~user4/Documents`

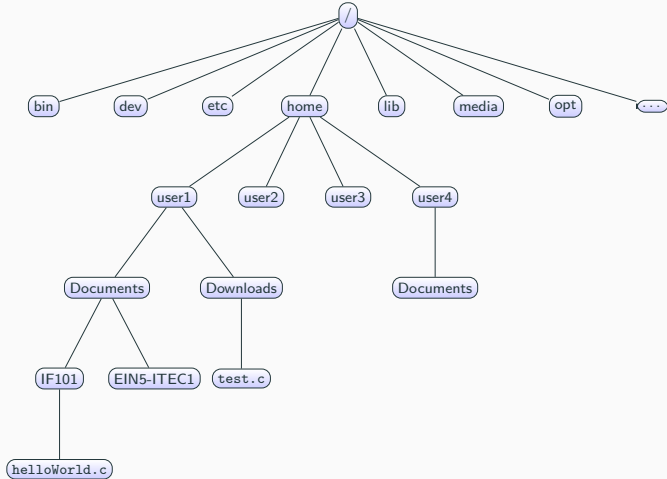
Chemins relatifs

Le point de départ **dépend de la position courante** (working directory) dans le système de fichier. Il n'est défini et valide que pour la position courante dans l'arborescence. Il commence par :

- `./` : Le répertoire courant
- `../` : Le répertoire parent
- `subdir1` : un fichier contenu dans le répertoire courant (équivalent à `./subdir1`)

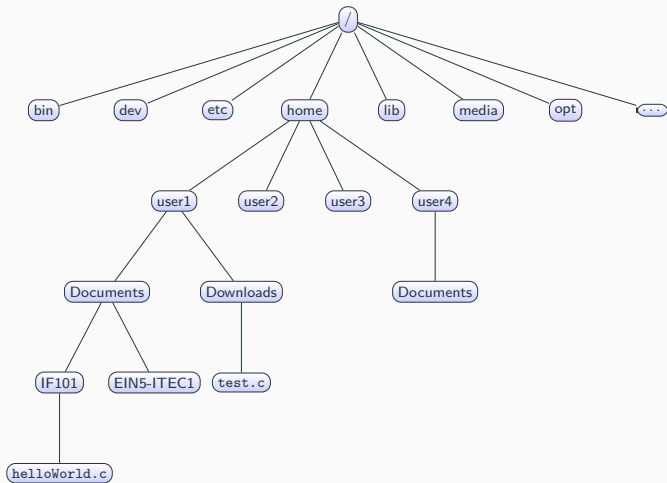


Chemins relatifs



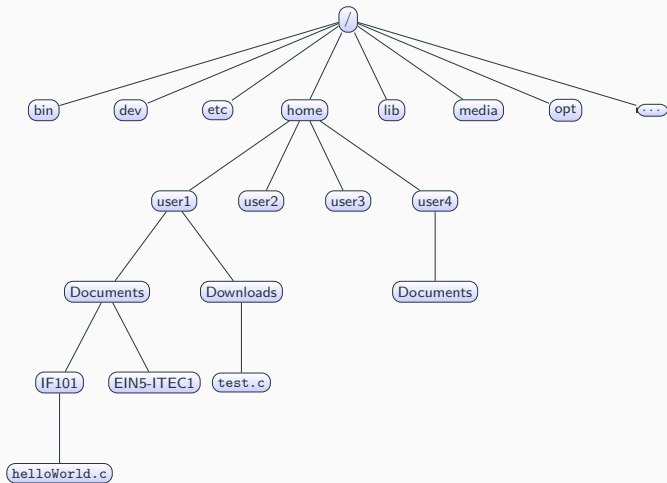
- Je suis l'utilisateur user1 et je suis dans mon répertoire de travail. Pour aller dans le répertoire IF101 je fais :
`cd Documents/IF101`

Chemins relatifs



- Je suis l'utilisateur `user1` et je suis dans le répertoire `Documents` de mon répertoire de travail. Pour aller dans le répertoire `Downloads` je fais :
`cd ../Downloads`

Chemins relatifs



- Je suis l'utilisateur user1 et je suis dans mon répertoire de travail. Pour aller dans le répertoire Documents depuis user4 je fais :
`cd ../user4/Documents/`

Quelques règles pour le nommage des fichiers

- pas d'espaces : utiliser "_" (underscore) ou "-" (minus)
espace est le caractère de séparation des mots-clés des commandes bash
- pas de caractères spéciaux : pouvant être interprété par le shell (\$, , ...)
- ajouter un suffixe pour désigner les types de fichiers : .txt, .html, .sh, .tgz, ...
- **L'extension ne définit pas le type de fichier!!!**
le nom d'un pdf peut très bien se terminer par «.txt», mais c'est pas une raison pour faire n'importe quoi.
- Utiliser la commande `file` pour connaître le type d'un fichier

Commandes de manipulation de fichiers

Se repérer

- `pwd` : afficher le répertoire courant
- `cd` : changer de répertoire courant
- `ls`, `ls -l`, `ls -la` : lister

Consulter

- `file` : déterminer le type
- `cat` : afficher un fichier
- `less` : parcourir un fichier
- `wc`, `wc -l`, `wc -w` : compter

Créer et modifier

- `mkdir` : créer un répertoire
- `touch` : créer un fichier vide ou changer sa date
- `cp`, `cp -r` : copier
- `mv` : déplacer ou renommer

Supprimer et protéger

- `rm`, `rm -r` : supprimer
- `chmod` : modifier les droits

Exercice : échauffement des doigts

Essayer la séquence de commandes suivantes et donner/interpréter les résultats :

- `cd`
- `pwd`
- `ls -la`
- `cd .`
- `pwd`
- `cd ..`
- `pwd`
- `ls -la`
- `cd ..`
- `pwd`
- `cd ~/mfaverge/cours/EIN5-ITEC1/Rugby/France/Bordeaux-Begles` (Utilisez TAB)
- `ls -la`
- `cat Liste_Matchs.csv`
- `cd ..`
- `pwd`
- `cd`
- `pwd`
- `cd -`
- `pwd`

Exercice : manipulation des fichiers

Après chacune des commandes suivantes, vérifier que la commande a été validée grâce à la commande `ls`

- Déplacez-vous dans votre home directory
- Créer un répertoire nommé `EIN5-ITEC1` et dans ce répertoire, un sous-répertoire `echauffement`
- Copier le fichier `~mfaverge/cours/EIN5-ITEC1/Rugby/France/Bordeaux-Begles/Liste_Matchs.csv` dans `echauffement`
- Copier ce nouveau fichier dans le même répertoire en le nommant `Liste.csv`
- Allez dans le répertoire `echauffement`
- Vérifier la présence des fichiers `Liste_Matchs.csv` et `Liste.csv`
- Renommer le fichier `Liste.csv` en `Liste.csv.bak`
- Afficher le contenu de ce `Liste.csv.bak`
- Remonter au répertoire parent de `echauffement`
- Copier `echauffement` en `echauffement.bak`
- Supprimer `echauffement`
- Créer le fichier `test` à l'intérieur de `echauffement.bak`
- Gardez votre dossier pour l'instant, nous nous en servons encore une fois.

Droits des fichiers

Toute opération sur un fichier est soumise à un ensemble de droits d'accès qui sont de 3 types.

r : droit de lecture

- Si répertoire, consultation de ses entrées (c.-à-d. ls autorisé)
- Sinon, consultation du contenu du fichier

w : droit d'écriture

- Si répertoire, droit de création, de renommage et de suppression d'une entrée dans le répertoire
- Sinon, droit de modification du contenu du fichier

x : droit d'exécution ou de traversée

- si répertoire, droit de traverser (c.-à-d. cd autorisé)
- sinon, droit d'exécution

- 3 catégories d'utilisateurs :
 - Propriétaire (u)
 - Groupe propriétaire (g)
 - Tous les autres (o)
- Chaque catégorie possède ses types d'accès `r w x`

Droits des fichiers - consultation

- `ls -ld` → donne les droits des fichiers
- Format de sortie de `ls -l`
- `- --- --- ---`
- `-` → Type du fichier
 - `-` : fichier régulier
 - `d` : répertoire
 - `l` : lien symbolique
 - `s` : socket
 - ...
- `---` → Droits du propriétaire (rwx)
- `---` → Droits du groupe (rwx)
- `---` → Droits des autres (rwx)

Droits des fichiers - modification

- Modification des droits sur un fichier/répertoire existant
 - `chmod droit fichier`
- Droits à appliquer :
 - Catégorie : u (=user), g (=group), o(=other), ou a (=all)
 - Opérations : + (ajout), - (retrait), = (affectation)
 - Exemple : u+rx,o=r
- Alternativement, on peut aussi les donner en octal : 4 (lecture), 2 (écriture) et 1 (exécution)
Exemple : 641 pour rw-r—x, 755 pour rwxr-xr-x

1. En se connectant à une machine de l'enseirb
 - Expliquer `-rw-----`
 - Expliquer `drwx--x--x` (cf `ls -ld ~mfaverge`)
 - Essayer d'afficher le contenu du fichier `~mfaverge/EIN5-ITEC1/file.txt` en utilisant TAB pour compléter son nom. Expliquez.
2.
 - À qui appartient le fichier `/etc/passwd` ?
 - Pourquoi n'avez-vous pas le droit de le modifier ?
 - Quels sont vos droits sur ce fichier ?
3. Donner deux façons possibles d'obtenir les droits suivants (en symbolique et en octal)
 - `-rw-r--r--`
 - `-rwxr-xr-x`
 - `-rwxr-xr--`
 - `-r--r--r--`
 - `-r----x---`

4. Soit le répertoire `drwxr-xr-x 2 user staff 68B Mar 23 12:01 tmp` Donner et expliquer le résultat des commandes suivantes sur `tmp` en supposant que `tmp` se trouve dans le répertoire personnel de `user`

- `chmod 355 tmp; ls tmp`
- `cd tmp; pwd`
- `cp /etc/passwd .; ls`
- `cd ..; chmod 755 tmp; ls tmp`
- `chmod 655 tmp; cd tmp`
- `chmod 555 tmp; rm -f tmp/passwd`

Faites les modifications nécessaires jusqu'à suppression de ce répertoire.

1. Revenez à votre répertoire `echauffement.bak` :
`cd ~/EIN5-ITEC1/echauffement.bak`
2. Enlever le bit de lecture sur le fichier `Liste.csv.bak` et vérifier à nouveau les droits
3. Afficher le contenu de ce `Liste.csv.bak`
4. Remettre le droit de lecture sur `Liste.csv.bak`
5. Revenir au répertoire parent
6. Enlever le bit autorisant le parcours sur le répertoire `echauffement.bak`
7. Essayer `ls echauffement.bak` puis `cd echauffement.bak`
8. Remettre les droits nécessaires, puis supprimer le répertoire `echauffement.bak`

Unix/Bash

Substitutions et variables

Substitutions et expansions (1/3)

Le shell analyse la ligne de commande avant de l'exécuter pour faire des substitutions et des expansions.

Expansion des accolades

`a{b,c,d}e` devient `abe ace ade`

Expansion des tildes

`~` → chemin vers votre répertoire personnel

`~toto` → chemin vers le répertoire personnel de toto

Expansion des variables

- syntaxe d'une variable : `$VAR`, `${VAR}`, `${VAR:-default}`
- affectation d'une variable :
`export VAR=value` → variable disponible dans l'environnement des processus enfants
`VAR=value` → variable disponible **uniquement** dans l'environnement du processus courant

Attention PAS d'espace entre le nom de variable et le =

Substitutions et expansions (2/3)

Substitution de commandes

`$( cmd )` exécute `cmd` dans un sous-shell et remplace `$( cmd )` par le résultat de l'exécution de la commande.

Coupure des mots

Chaque séquence de blancs (espaces, tabulations, retours de ligne) délimite les mots-clés de la ligne de commande : commande, paramètres, options, variables, ...

Point important

- `'une chaîne de caractères'` : le contenu n'est pas interprété
- `"une chaîne de caractères"` : le contenu est interprété

Substitutions et expansions (3/3)

Expansion des noms de fichiers

Le shell recherche les caractères spéciaux suivants avant d'exécuter la commande : *, ?, [...].

- * : n'importe quelle chaîne de caractères, même vide ;
- ? : n'importe quel caractère ;
- [...] : n'importe quel caractère spécifié entre crochets.

Dans les crochets, on peut indiquer :

- des caractères seuls ;
- des ensembles de caractères séparés par un - (si - est le premier caractère, il désigne le caractère - lui-même).

Exemple : [04a-dG-L6] représente l'ensemble des caractères composé de :
0, 4, a, b, c, d, G, H, I, J, K, L, 6

Protection

Le caractère d'échappement \ protège un caractère spécial lorsqu'il ne doit pas être évalué.

Par exemple : mkdir my\ dir

Exercices

- Que fait `echo projet/{src/{module1,module2},lib,bin}` ?
- Que produit la séquence suivante ?
`echo ${BROL:-Je ne le connais pas}`
`export BROL="Monsieur Brol"`
`echo ${BROL:-Je ne le connais pas}`
- Que produit `echo $PATH` ? Que représente cette variable d'environnement ?
- Que fait la séquence suivante : `export VAR=$(date); echo $VAR` ?
- Que fait la séquence suivante : `export BROL="Monsieur Brol"`
`echo "$BROL et trol"`
`echo '$BROL et trol'`
- Que fait `ls -l /usr/lib/lib[a-jt-z]??e*`
- Se placer dans le répertoire `~mfaverge/cours/EIN5-ITEC1/Rugby/` (Utiliser C-r et l'historique)
 - Avec `echo` ou `ls`, trouver en une ligne le seul joueur né en 2003
 - Comment lister tous les joueurs nés en 1995 et 1999 ?
- Que fait la séquence suivante : `RAC=/bin`
`echo $RAC`
`echo $RAC/pwd`
`$RAC/pwd`

- Que fait la séquence suivante : `echo /bin/[a-c]?`
- Que fait la séquence suivante : `echo '/bin/[a-c]?'`
`echo "/bin/[a-c]?"`
`N='/bin/[a-c]?'`
`echo $N`
- Que fait la séquence suivante : `A=un`
`bash; echo /$A/`
`^D`
`export A; bash; echo /$A/`
`A=deux; B=trois; echo /$A/$B`
`^D`
`echo /$A/$B`

Note

`^D` peut aussi être noté `C-D` ou `Ctrl-D` et est équivalent à `exit`.

Unix/Bash

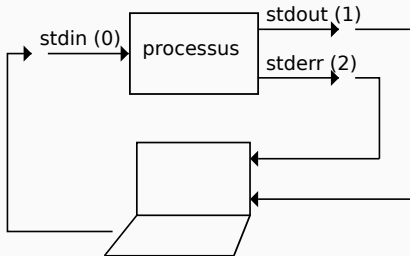
Mécanismes d'entrées/sorties

Mécanismes d'entrées/sorties

Généralement une opération d'E/S :

- transfert de données entre une zone mémoire et un fichier ou un périphérique
- Ex. un processus effectue des E/S : depuis le clavier vers l'écran

À sa création, chaque processus possède 3 flots E/S standards.

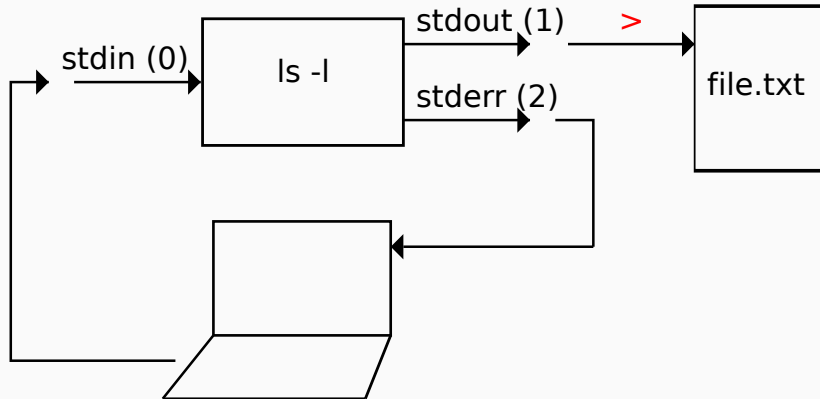


- `stdin (n° 0)` : entrée standard
opérations lecture, par défaut le clavier
- `stdout (n° 1)` : sortie standard
opérations d'écriture, par défaut l'écran
- `stderr (n° 2)` sortie d'erreur
opérations d'affichage des messages
d'erreurs, par défaut l'écran

Unix permet à l'utilisateur de modifier et de reconnecter les flots d'E/S
c'est le mécanisme de redirection

Mécanismes de redirection (1)

Rediriger un processus (programme) vers un fichier via '>'.
Ex : `ls -l > file.txt`

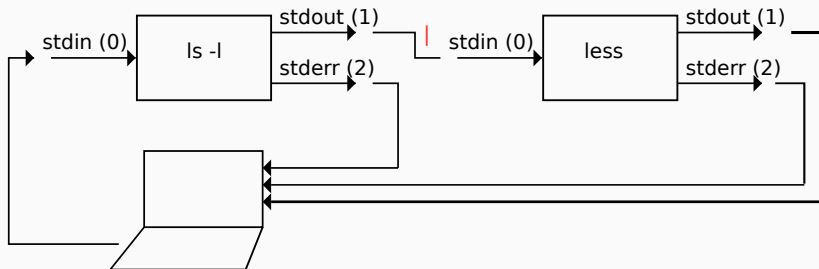


Si le fichier existe il est écrasé

Mécanismes de redirection (2)

Rediriger un processus (programme) vers un autre processus (programme) via le pipe (tube) : ' | '

Ex. `ls -l | less`



Possibilité de créer des pipelines complexes en une seule ligne de commande

Mécanismes de redirection (3)

Redirections courantes

- `< file` : redirection de l'entrée standard depuis un fichier
- `> file` : redirection de la sortie standard vers un fichier
- `>> file` : redirection en ajoutant à la fin du fichier
- `cmd | cmd` : pipe entre deux commandes

Sortie d'erreur

- `2> file` : redirige la sortie d'erreur vers file
- `2>&1` : fusionne la sortie d'erreur avec la sortie standard

Exercices

1. Que fait la ligne de commande suivante ?

```
date > tmp.txt; ls -l ~/.??* | wc -l >> tmp.txt
```

Que contient finalement tmp.txt

2. Que fait la ligne de commande suivante ?

```
echo $(date) : $(who | wc -l) util. sur $(hostname)
```

3. Rediriger la sortie de `ls -la` vers un fichier

4. Placez-vous dans votre répertoire personnel

- Déplacez-vous dans le répertoire `repertoire_inexistant`, un message d'erreur est-il apparu ? si oui où ?
- Recommencer mais en redirigeant la sortie d'erreur vers le fichier `err`
- Un message est-il apparu ? le fichier `err` a-t-il été créé ? Que contient le fichier `err` ?
- Recommencer mais cette fois en ajoutant à la fin (`>>`). Que contient le fichier `err` ?

UNIX Reference Card

Warnings!!

When a file has been **DELETED** it can only be restored from a backup. The original is gone!

When a file is **OVERWRITTEN** it has been changed forever! It can only be restored from a backup.

Directory Abbreviations

~	home directory (tilde)
~username	another user's home directory
.	current working directory
..	parent of current working directory

Communication

ssh [options] hostname

Ssh (Secure Shell) a program for logging into a remote host. Replaces telnet, rlogin, and rsh

options:

-l login_name	specifies the user to log in on the remote machine
---------------	--

scp [options] user@host1:file1 user@host2:file2

Secure copy files between hosts on a network; uses ssh for data transfer.

options:

-p	preserve modification times
-r	recursively copy entire directories

Comparison

diff [options] file1 file2

Compare two text files.

options:

-a	treat all files as text files
-b	ignore repeating blanks and end-of-line blanks; treat successive blanks as one
-i	ignore case in text comparison
-q	output only whether files differ

File Management

cat [options] [files]

Read one or more *files* and print them on standard output. Use the > operator to combine several files into a new file; use >> to append files to an existing file.

options:

-n	print the number of the output line to the line's left
-s	squeeze out extra blank lines

cd [dir]

Change working directory to dir; default is the users home directory.

chgrp newgroup files

Change the group of one or more *files* to *newgroup*. *newgroup* is either a group ID number of a group name. Only the owner can change the group.

options:

-c	print information about those files that are affected
-R	recursively apply changes to subdirectories

chmod [options] mode files

Change the access *mode* (permissions) of one or more *files*. Only the owner of a file or a privileged user may change its mode.

options:

-c	print information about affected files
-R	recursively apply changes to subdirectories

mode:

can be numeric

4	read
2	write
1	execute

or an expression of the form *who opcode permission*. *who* is optional (if missing, default is **a**)

who

u	user
g	group
o	other
a	all (default)

opcode

+	add permission
-	remove permission
=	assign permission

permission

r	read
w	write
x	execute
X	set execution permission only if executable by user

cp [options] file1 file2

cp [options] files directory

Copy *file1* to *file2*, or copy one or more *files* to the same names under *directory*.

options:

-a	preserves attributes of original files
-f	remove existing files in the destination
-i	prompt before overwriting destination files
-r	recursively copy directories
-s	make symbolic links instead of copying

file [options] files

Classify the named *files* according to the type of data they contain.

less [options] [filename]

A program for browsing or paging through files or other output. Can use arrow keys for scrolling forward or backward.

options:

see **man** pages for options (type: **man less**)

ln [options] sourcename [destname]

ln [options] sourcenames destdirectory

Create links for files, allowing them to be accessed by different names.

options:

-b	backup files before removing originals
-i	prompt for permission before removing files
-s	create a symbolic link. This lets you see the name of the link when you run ls -l (otherwise there is now way to know the name that a file is linked to).

ls [options] [names]

List the contents of a directory. If no *names* are given, the files in the current directory are listed.

options:

-a	list all files, including hidden files
-c	list files by status change time
-l	long format listing (permissions, owner, size, modification time)

mkdir [options] directories

Create one or more directories.

options:

-m mode	set the access <i>mode</i> for new directories. See chmod for <i>mode</i> formats.
-p	create intervening parent directories if they don't exist

more [options] [files]

Display the content of the named *files* one screen at a time. See less for an alternative.

options:

see **man** pages for options (type: **man more**)

pwd

Print the full pathname of the current working directory.

scp [options] user@host1:file1 user@host2:file2

Secure copy files between hosts on a network; uses ssh for data transfer.

options:

-p	preserve modification times
-r	recursively copy entire directories

mv [options] sources target

Move or rename files and directories. The *source* and *target* determine the result.

source	target	result
file	name	rename file as <i>name</i>
file	existing	overwrite existing file
	file	with source file
directory	name	rename directory as
	name	
directory	existing	move directory to be a
directory	subdirectory of	
existing directory		

options:

-b	back up files before moving
-f	force the move
-i	query user before removing files

rm [options] files

Delete one or more files. *Once a file or directory has been removed it can only be retrieved from a backup!*

options:

- d remove directories, even if they are not empty
- f remove files without prompting
- i prompt for file removal
- r recursively remove an entire directory and its contents, including subdirectories. *Be very careful with this option.*

Miscellaneous

! Repeat the last command

!string Repeat the last command beginning with *string*.

cal [-jy] [[month] year]

Print a 12-month calendar for the given *year* or a one-month calendar of the given *month* and year. No arguments, print a calendar for the current month.

options:

- j display Julian dates
- y display entire current year

clear

Clear the terminal display

history

Display list of most recently executed commands

kill [option] IDs

Send a signal to terminate one or more process *IDs*.

options:

- l list all signals
- signal the signal number (from **ps -f**) or name (from **kill -l**). You can kill just about any process with a signal number of 9.

man command

Display information from the online reference manuals.

jobs [options] job_id

Display status of jobs in the current session. Simply specifying jobs returns the status of all stopped jobs, running background jobs, and all suspended jobs.

options:

- l provide more information about each job listed
- p display only the process IDs for the process group leaders of the selected jobs

whereis command

Locate a *command*; display the full pathname for the *command*.

which [commands]

List which files would be executed if the named *commands* had been run.

Searching

egrep [options] [regex] [files]

grep [options] [regex] [files]

Search one or more *files* for lines that match a regular expression *regex*. To include characters such as +, ?, |, (), blank spaces, etc. enclose these expressions in quotes. See **man** pages for the differences between **egrep**, **fgrep**, and **grep**.

options:

- c print only a count of matched lines
- i ignore case
- l list filenames but not matched lines
- n print lines and their line numbers
- v print all lines that do not match *regex*

find [pathnames] [conditions]

Useful for finding particular files. **find** descends the directory tree beginning at each *pathname* and locates files that meet the specified *conditions*.

options:

- name *pattern* find files whose name matches *pattern*
- print print the matching files and directories using their full pathnames

see **man** pages for options (type: **man find**)

Storage

compress [options] [files] – compress file

uncompress [options] [files] – uncompress compressed file

compress reduces the size of the named *files*. When possible the resulting compressed file will have the file extension **.Z**. Compressed files can be restored using **uncompress**.

options:

- d uncompress file, same as **uncompress**
- v prints the percentage reduction
- V prints the version of compress

gzip [options] [files] – compress file

gunzip [options] [files] – uncompress gzipped file

GNU compression utility. Renames compressed files *filename.gz*. Uncompress with **gunzip**.

options:

- d uncompress file, same as **gunzip**
- r recursively compress or decompress files within a directory
- v print name and percent size reduction for each file

tar [options] [tarfile] [other-files]

Copy *files* to or restore *files* from an archive. If any files are directories, **tar** acts on the entire subtree.

options:

- c create a new archive
- d compare the files stored in tarfile with other-files
- r append other-files to the end of an existing archive
- t print the names of files in archive
- v verbose, print filenames as they are added or extracted
- x extract *other-files* from archive, or extract all files if *other-files* not specified

System Status

Control-C

Stop (interrupt) job running in the foreground

Control-Z

Suspend job running in the foreground

date [options] [+format] [date]

Print the current date and time. You may specify a display *format*.

options:

see **man** pages for options (type: **man date**)

df [options] [name]

Report the amount of free disk space available on all mounted file systems or on a given *name*.

options:

- k print sizes in kilobytes

du [options] [directories]

Print disk used by each named directory and its subdirectories.

options:

- k print sizes in kilobytes
- s print only the grand total for each directory

env [option] [variable=value ...] [command]

Display the current environment or, if an environment *variable* is specified, set it to a new *value* and display the modified environment.

option:

- u unset the specified *variable*

ps [options]

Report on active processes.

options:

- a list all processes except processes not associated with the terminal
- e list all processes
- l produce long format listing
- u *list* list for usernames in *list*

quota [option]

Display disk usage and limits

option:

- v report quotas even if they haven't been exceeded

Contact Information

Phone: 612-626-0802

Email: help@msi.umn.edu

WWW: http://www.msi.umn.edu

http://www.msi.umn.edu/user_support